12

Лекция 27–28 апреля

Разрешимость и теорема Райса

Константин Чухарев

Дискретная математика — Весна 2027

Три класса языков

> *Мы должны знать. Мы будем знать.*
>
> — ***Давид Гильберт***

Разрешитель: ответ на каждом входе

Хочется инструмент, который по описанию программы и её входу заранее говорит, завершится ли она.

Определение 1 (Разрешимость (*decidability*))

Язык L **разрешим**, если существует машина Тьюринга D, которая на любом входе w останавливается и принимает его тогда и только тогда, когда $w \in L$.

Отрицательный ответ — тоже остановка: зациклиться разрешителю нельзя.

Пример (Разрешимый язык)

Язык $a^n b^n$ разрешим: машина вычёркивает крайние пары ab.

Вход aabb: aabb $\to$ aXbX $\to$ XX — принято за два прохода.

Язык без разрешителя тоже существует: HALT распознаваема, но неразрешима — диагональное доказательство ниже.

Распознаватель: гарантия только для своих

Симуляция даёт «да» надёжно, а «нет» — никак: зацикливание не наблюдаемо.

Определение 2 (Распознаваемость (*recursively enumerable*))

Язык L **распознаваем**, если существует машина Тьюринга, которая принимает каждый вход из L.

На входах вне L распознаватель может зациклиться — определение не обязывает его останавливаться.

Пример («HALT» распознаваема)

U симулирует M на w и принимает при остановке симуляции — на чужих словах симулятор крутится вечно.

Вся сила распознавателя — предъявление положительных свидетелей, и только.

Ко-распознаватель: гарантия для чужих

У симуляции есть зеркальный двойник: машина, отвечающая за слова вне языка.

Определение 3 (Ко-распознаваемость (*co-recognizability*))

Язык L **ко-распознаваем**, если распознаваем его дополнение $\overline{L}$.

Распознаватель дополнения принимает каждый «нет»-вход и может зациклиться на каждом «да»-входе.

Пример (Дополнение «HALT»)

$\overline{\text{HALT}}$ — пары $(\langle M \rangle, w)$, на которых M зацикливается.

«Да»-свидетель здесь — бесконечность пробега, а бесконечность не предъявить за конечное время.

Распознаваемость и ко-распознаваемость

Два свойства независимы: у языка может оказаться только одна из двух сторон.

Язык	Распознаваемость	Ко-распознаваемость
$a^n b^n$	есть	есть
HALT	есть	нет
$\overline{\text{HALT}}$	нет	есть

Разрешимый язык несёт обе стороны: инверсия ответа разрешителя — готовый ко-распознаватель.

Дополнение не наследуется
Симуляция даёт конечный принимающий прогон, бесконечный пробег не предъявить — у HALT и её дополнения ровно по одной стороне. Распознаваемость не наследуется дополнением: про чужие слова распознаватель ничего не знает.

Перечислитель: та же сила другим жестом

Определение 4 (Перечислимость (*enumerability*))

Язык L **перечислим**, если есть МТ E — **перечислитель**, — печатающая все слова L, каждое хотя бы раз и ничего кроме.

Одна сила печатает, другая проверяет — теорема уравнивает.

Пример (Перечислитель для «HALT»)

На этапе s машина E делает по s шагов каждой симуляции из первых s машин $M_1, ..., M_s$ на первых s входах $w_1, ..., w_s$.

Остановилась симуляция пары $(\langle M_i \rangle, w_j)$ — E печатает $\langle M_i \rangle w_j$ и к паре больше не возвращается. Пара (i, j) дожидается своего этапа $s \geq \max(i, j)$, поэтому каждая останавливающаяся пара напечатана.

Зацикливающиеся пары не печатаются никогда — перечислен в точности HALT.

Перечислимость и распознаваемость

Теорема связывает две интуиции: печать и проверку.

Теорема 1 (Перечислитель равносилен распознавателю)

L перечислим тогда и только тогда, когда L распознаваем.

Доказательство

Докажем эквивалентность в обе стороны.

Пусть сначала E — перечислитель L. Распознаватель запускает E и принимает, как только w появился на печати: чужие слова не появятся никогда.

Обратно: чередуем шаги распознавателя M по всем входам и печатаем вход в момент принятия.

Каждое слово из L будет принято и напечатано, слова вне L — нет.

Разрешимость с двух сторон

Ко-распознаватель — вторая половина разрешителя: две половины складываются в целое.

Теорема 2 (Разрешимость равносильна распознаваемости с двух сторон)

L разрешим тогда и только тогда, когда распознаваемы L и $\overline{L}$.

Доказательство

Докажем эквивалентность в обе стороны. Пусть L разрешим: разрешитель D служит распознавателем L, а инверсия его ответа — распознавателем $\overline{L}$.

Обратно: распознаватели M и N запускаем параллельно, чередуя шаги — ровно один язык содержит вход, и его приём — верный ответ.

Для HALT вторая половина отсутствует — ниже это станет теоремой.

Замкнутость разрешимых

Класс ценен тогда, когда операции не выводят из него.

Теорема 3 (Замкнутость разрешимых языков)

Разрешимые языки замкнуты относительно объединения, пересечения и дополнения.

Доказательство

Докажем построением: для объединения и пересечения запускаем D_1 и D_2 по очереди и соединяем ответы через $\vee$ и $\wedge$.

Дополнение: инверсия ответа D.

Пример (Пример на простоту)

Язык записей простых чисел разрешим перебором делителей до $\sqrt{n}$: $n = 91$ даёт делитель 7, $n = 97$ не имеет делителей среди $2, \ldots, 9$, дополнение — инверсией ответа.

Замкнутость распознаваемых

У распознавателей одна гарантия — приём своих слов, поэтому и замкнутость неполная.

Теорема 4 (Замкнутость распознаваемых языков)

Распознаваемые языки замкнуты относительно объединения и пересечения.

Замкнутость относительно дополнения у класса отсутствует — контрпримером служит проблема остановки.

Доказательство

Объединение: чередуем шаги M_1 и M_2 и ждём первого приёма.

Пересечение: запускаем M_1, затем M_2 — для слова из пересечения обе машины принимают за конечное время.

Конкретное пересечение: пары, на которых остановятся обе машины.

Проблема остановки

> *Машины удивляют меня с завидной регулярностью.*
>
> ***— Алан Тьюринг***

Универсальная машина: программа как данные

Диагональ опирается на умение машины исполнять другие машины по описанию.

Теорема 5 (Универсальная машина)

Существует машина Тьюринга U, которая по паре $(\langle M \rangle, w)$ симулирует работу M на входе w.

Доказательство

Докажем построением: кодировка $\langle M \rangle$ — конечная строка с таблицей переходов.

U держит конфигурацию симулируемой машины на ленте и выполняет шаги по таблице — каждый шаг M рано или поздно воспроизводится.

Самоприменение законно: $\langle M \rangle$ — такая же строка, как w.

Интерпретаторы и виртуальные машины — инженерная жизнь теоремы.

Проблема остановки

Теперь вопрос, который диагональ отправит в неразрешимые: завершится ли программа?

Определение 5 («HALT»)

По паре $(\langle M \rangle, w)$ узнать, останавливается ли M на входе w:

$$\mathrm{HALT} = \{\langle M \rangle w \mid M \text{ останавливается на } w\}.$$

Пример (Два конкретных запроса)

Машина M_{loop} при любом символе остаётся в q_0: $(q_0, a) \to (q_0, a)$ — тогда $\langle M_{\text{loop}} \rangle 0 \notin \mathrm{HALT}$.

Машина M_{acc} переходит в состояние принятия при любом символе — тогда $\langle M_{\text{acc}} \rangle \varepsilon \in \mathrm{HALT}$.

Диагональная машина D

Допустим, существует разрешитель H: на входе $\langle M \rangle w$ она всегда останавливается и отвечает в точности по остановке M.

Строим машину D:

1. На входе x запускает H на паре (x, x) — спрашивает, остановится ли машина с кодом x на собственном коде.
2. Если H приняла — зацикливается.
3. Если H отвергла — принимает.

Почему собственный код — не фокус

Круга в построении нет: D строится после H, а строка $\langle D \rangle$ существует, как код любой машины.

Вопрос «что сделает D на $\langle D \rangle$» задаётся после построения D: анализатор подаёт машине её собственный текст, как компилятор читает сам себя.

Подстановка: D на собственном коде

Остался один ход — подставить D в самое себя.

Теорема 6 (Неразрешимость проблемы остановки)

Язык HALT неразрешим.

Доказательство

Докажем от противного: пусть H решает HALT, D построена, рассмотрим $D(\langle D \rangle)$.

Если D остановилась, сработала ветка «отвергла» — H ответила «зациклится», но D остановилась.

Если D зациклилась, сработала ветка «приняла» — H ответила «остановится», но D зациклилась.

Оба случая делают H ошибочной — противоречие.

Распознаваема, но не разрешима

Диагональ запретила решатель с «да» и «нет», а симуляция даёт лишь «да».

Теорема 7 («HALT» вне разрешимых)

HALT распознаваема, но не разрешима, а её дополнение даже не распознаваемо.

Доказательство

Распознаваемость. Симуляция: U принимает $\langle M \rangle w$ в момент остановки M и в противном случае крутится вместе с ней.

Нераспознаваемость дополнения. Пусть N распознаёт $\overline{\mathrm{HALT}}$ — на входе $\langle M \rangle w$ запустим U и N параллельно, чередуя шаги.

Ровно одна машина примет: U — если M остановится, N — если зациклится.

Её приём — верный ответ на вопрос остановки — противоречие с диагональю.

Карта четырёх классов

Каждая неразрешимость уточняет карту: соберём классы и их замкнутости в одну таблицу.

Класс	Машина	Объединение	Пересечение	Дополнение
Регулярные	ДКА	да	да	да
Контекстно-свободные	МП-автомат	да	нет	нет
Разрешимые	разрешитель	да	да	да
Распознаваемые	распознаватель	да	да	нет

Клетка «нет» в последней строке — теорема о дополнении HALT: у языка есть распознаватель, а у дополнения его нет.

Сведения

> *Лучше решить правильную задачу неправильным способом, чем неправильную задачу правильным.*
>
> ***— Алан Перлис***

Сведение: перенос вопроса

Диагональ строят один раз — дальше её переиспользуют.

Определение 6 (Сведение (*many-one reduction*))

Язык A **сводится** к B, пишется $A \leq_m B$, если есть вычислимая тотальная функция f с условием

$$w \in A \leftrightarrow f(w) \in B.$$

Пример (От остановки к пустому входу)

Цель — $\mathrm{HALT}_\varepsilon$: описания машин, останавливающихся на пустом входе. Отображение $f(\langle M \rangle, w) = \langle M' \rangle$: M' на любом входе записывает w и дальше работает как M.

M остановилась на w — тогда M' подготовила себе вход и остановилась: $\langle M' \rangle \in \mathrm{HALT}_\varepsilon$. M зациклилась — M' крутится после записи: $\langle M' \rangle \notin \mathrm{HALT}_\varepsilon$.

Логика переноса

Из определения сведения сразу видно, как переносятся алгоритмы и их отсутствие.

Теорема 8 (Что переносит сведение)

Пусть $A \leq_m B$.

- Если B разрешима, то разрешима и A.
- Если A неразрешима, то неразрешима и B.
- Если B распознаваема, то распознаваема и A.
- Если B ко-распознаваема, то ко-распознаваема и A.

Доказательство

Разрешимость: разрешитель B на образе $f(w)$ отвечает и про w.

Неразрешимость — контрапозиция, распознаваемость и ко-распознаваемость — та же подстановка f для дополнений.

Куда идёт сведение

Теорема читается в одну сторону, и путаница в направлении — частая ошибка.

Ошибка направления

Транзитивность работает в одну сторону: из $A \leq_m B$ и неразрешимости A следует неразрешимость B. Сведение $B \leq_m A$ такого вывода не даёт.

Пустота языка

Первая остановка на маршруте сведений — вопрос о пустоте языка машины.

Пример (От HALT к E_{TM})
$E_{\mathrm{TM}} = \{\langle M \rangle \mid L(M) = \varnothing\}$.

Сводящая функция $f(\langle M \rangle, w) = \langle M' \rangle$: M' на любом входе игнорирует его, симулирует M на w и принимает при остановке симуляции.

M остановилась на w — тогда $L(M') = \Sigma^*$, зациклилась — тогда $L(M') = \varnothing$.

Разрешитель E_{TM} с инверсией ответа решал бы HALT — значит, E_{TM} неразрешима.

Эквивалентность программ

При рефакторинге первым делом спрашивают, вычисляют ли две версии кода одно и то же.

Теорема 9 (Неразрешимость эквивалентности)

Язык $\mathrm{EQ}_{\mathrm{TM}} = \{\langle M_1 \rangle \langle M_2 \rangle \mid L(M_1) = L(M_2)\}$ неразрешим.

Доказательство

Докажем сведением пустоты: $f(\langle M' \rangle) = \langle M' \rangle \langle M_{\mathrm{loop}} \rangle$, где M_{loop} не принимает ничего.

Язык M' пуст $-$ $L(M') = \varnothing = L\big(M_{\mathrm{loop}}\big)$ и образ лежит в $\mathrm{EQ}_{\mathrm{TM}}$. Язык непуст $-$ образ вне.

Равносильность $\langle M' \rangle \in E_{\mathrm{TM}} \leftrightarrow f(\langle M' \rangle) \in \mathrm{EQ}_{\mathrm{TM}}$ установлена: решатель эквивалентности решал бы пустоту $-$ противоречие.

Соответствия Поста: определение

Сведения работают не только внутри мира машин: разберём задачу про строки и кости.

Определение 7 (Соответствие Поста (*Post correspondence*))

Дан конечный набор костей: на верхней половине каждой — строка t_i, на нижней — строка b_i.

Спросить: существует ли непустая последовательность костей, у которой верхняя цепочка совпадает с нижней?

Пример (Совпадение и его отсутствие)

Набор из двух костей: у b_1 верх 1 и низ 111, у b_2 верх 11111 и низ 1.

Последовательность $b_1 b_1 b_2$ даёт сверху и снизу по семь единиц — совпадение 1111111.

Одна кость с верхом 1 и низом 11 совпадений не имеет: k единиц сверху против $2k$ снизу неравны при $k \geq 1$.

Соответствия Поста: неразрешимость

Машины сводятся даже к костям домино.

Теорема 10 (Неразрешимость соответствий Поста)

Язык PCP неразрешим.

Доказательство

Докажем сведением: по паре $(\langle M \rangle, w)$ строим набор костей. Стартовая кость ставит рядом $q_0 w$ и её копию, кости-шаги ведут конфигурации по таблице M, заполнители дотягивают короткую сторону.

Пока вычисление не остановилось, низ длиннее верха на недостроенный шаг, остановилась — финальная кость закрывает принимающую конфигурацию.

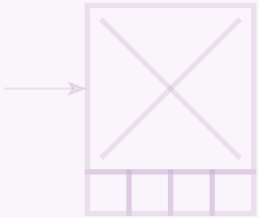
Итого совпадение существует в точности при $\langle M \rangle w \in \mathrm{HALT}$: решатель PCP решал бы HALT — противоречие.

Цепочка неразрешимостей

Задача	Вопрос	Откуда неразрешимость
HALT	останавливается ли M на w?	диагональный аргумент
$\mathrm{HALT}_\varepsilon$	останавливается ли M на пустом входе?	$\mathrm{HALT} \leq_m \mathrm{HALT}_\varepsilon$
E_{TM}	пуст ли язык машины?	$\mathrm{HALT} \leq_m E_{\mathrm{TM}}$
$\mathrm{EQ}_{\mathrm{TM}}$	совпадают ли языки двух машин?	$E_{\mathrm{TM}} \leq_m \mathrm{EQ}_{\mathrm{TM}}$
PCP	есть ли совпадение у набора костей?	$\mathrm{HALT} \leq_m \mathrm{PCP}$

Каждая стрелка таблицы — явная вычислимая функция, которую можно запрограммировать.

Классы расходятся: HALT, $\mathrm{HALT}_\varepsilon$ и PCP распознаваемы и не разрешимы, E_{TM} — ко-распознаваема и не распознаваема, $\mathrm{EQ}_{\mathrm{TM}}$ не лежит ни в одном из двух классов.



Теорема Райса

> *Остерегайтесь ошибок в приведённом выше коде: я лишь доказал его корректность, но не исполнил его.*
>
> **— Дональд Кнут**

Свойство языка или свойство машины?

Пустота, эквивалентность, универсальность — каждое свойство потребовало своего сведения.

Определение 8 (Экстенсиональное свойство)

Свойство P описаний машин **экстенсионально**, если оно зависит только от языка:

$$L(M_1) = L(M_2) \quad \Rightarrow \quad (\langle M_1 \rangle \in P \leftrightarrow \langle M_2 \rangle \in P).$$

Свойство **нетривиально**, если существуют машины и с P, и без P.

Пример (Экстенсионально или нет)

«Язык пуст» — экстенсионально и нетривиально.

Не экстенсиональны «не более 100 состояний» и «принимает ли M слово 01 за сто шагов».

Теорема Райса

Отдельные сведения для каждого свойства — бесконечная работа: Райс обрывает её одним утверждением.

Теорема 11 (Райс, 1953)

Всякое нетривиальное экстенсиональное свойство распознаваемых языков неразрешимо.

Иначе говоря: для каждого нетривиального P язык $\{\langle M\rangle \mid \langle M\rangle \in P\}$ неразрешим.

Пример (Что закрывается одним ударом)

Пуст ли язык? Конечен ли? Регулярен ли? Содержит ли слово 01? Совпадает ли с Σ^*?

Каждое свойство экстенсионально и нетривиально — каждое неразрешимо.

Разрешимы лишь тривиальные свойства: выполненные для всех языков или ни для одного.

Доказательство Райса

Схема повторяет сведение к пустоте языка.

Доказательство (Теоремы Райса)

Докажем сведением HALT к P.

Без ограничения общности пустой язык не обладает P: иначе берём дополнение свойства, разрешимость у них общая.

Нетривиальность даёт машину A, чей язык свойством обладает: $L(A) \in P$.

Строим M': на входе x она игнорирует x, симулирует M на w, затем работает как A.

M остановилась на w — тогда $L(M') = L(A)$ и $\langle M' \rangle \in P$, зациклилась — тогда $L(M') = \varnothing$ и $\langle M' \rangle \notin P$.

Разрешитель P на $\langle M' \rangle$ отвечает на вопрос остановки — противоречие.

Программа-подмена игнорирует вход: язык M' зависит только от остановки M на w.

Сведение на конкретном свойстве

Пример (Свойство «пустая строка в языке»)

Свойство P: $\varepsilon \in L(M)$.

Экстенсиональность: язык — в точности множество принимаемых слов, и принадлежность ε зависит лишь от языка.

Нетривиальность: машина, принимающая всё, свойство имеет, машина M_{loop} — нет.

По паре $(\langle M \rangle, w)$ строим M' из доказательства: симулируем M на w, при остановке принимаем.

M остановилась — M' принимает каждое слово, включая ε — свойство есть, зациклилась — M' не принимает ничего — свойства нет.

Где кончается теорема Райса?

Три свойства вне досягаемости

«У машины не более 100 состояний» — не экстенсионально и к тому же разрешимо: число читается из кодировки.

«Останавливается ли M на пустом входе за сто шагов» — не экстенсионально и разрешимо: достаточно симуляции ста шагов.

«Останавливается ли M на пустом входе» — не экстенсионально, и теорема Райса его не закрывает: неразрешимость доказывается отдельным сведением.

Граница Райса — граница экстенсиональности, а не граница неразрешимости.

За границей живут и разрешимые свойства (число состояний), и неразрешимые (поведение на входе).

Инженерные следствия

Пример (Три инструмента и их пределы)

Верификатор завершения отвечает на вопрос тотальности — неразрешимость тотальности доказана сведением.

Проверка рефакторинга отвечает на вопрос эквивалентности старого и нового кода — неразрешимость эквивалентности доказана сведением.

Идеальный антивирус по программе V, выполняющей вредоносный код при остановке симулируемой M, решал бы HALT.

Универсального верификатора и идеального антивируса не существует: оба решали бы нетривиальные свойства поведения программ.

Инструменты выживают третьим ответом — «да», «нет» или «не знаю».

За пределами машин Тьюринга

> *Любая достаточно развитая технология неотличима от магии.*
>
> ***— Артур Кларк***

Оракулы

Насколько неразрешимость «трудна»? Задачи сравнивают через гипотетических помощников.

Определение 9 (МТ с оракулом)

МТ с дополнительной лентой-оракулом: записав строку и перейдя в состояние запроса, машина за один шаг узнаёт, принадлежит ли строка языку A.

Оракул — чёрный ящик, решающий A: подпрограмма, отвечающая мгновенно.

Пример (Оракул для «HALT»)

Даже он не решает тотальность: вопрос «для всех ли x машина останавливается?» требует бесконечной серии запросов — по одному на каждый вход.

Лестница иерархии

Неразрешимость не одинакова: логическая сложность определения делит её на уровни.

Уровень	Задача	Кванторная форма
Δ_1	разрешимые задачи	ответ без неограниченных кванторов
Σ_1	проблема остановки	$\exists t.$ остановится за t шагов
Π_1	дополнение остановки	$\forall t.$ не остановится за t шагов
Σ_2	конечность языка	$\exists n \forall w.$ слово длиннее n не принимается
Π_2	тотальность	$\forall x \exists t.$ остановится на каждом x за t шагов
Σ_3	коконечность языка	$\exists \forall \exists.$ отвергается лишь конечное число слов

Кванторная глубина — мост к логике первого порядка.

Оракул для HALT решает Σ_1-вопросы, а Π_2-тотальность требует оракула уровнем выше — иерархия не кончается.

Функция занятого бобра

Невычислимое встречается и без оракулов: максимум по конечному множеству программ.

Определение 10 (Занятый бобер (*busy beaver*))

$\Sigma(n)$ — максимум единиц, которые запишет останавливающаяся машина с n состояниями и алфавитом $\{0, 1\}$ на пустой ленте, а $S(n)$ — максимум её шагов.

Σ мажорирует любую вычислимую функцию: для всякой вычислимой f выполнено $\Sigma(n) > f(n)$ при достаточно больших n.

Знание $S(n)$ решало бы остановку: машина, не остановившаяся за $S(n)$ шагов, не остановится никогда.

Известны $\Sigma(1) = 1$, $\Sigma(2) = 4$, $\Sigma(3) = 6$, $\Sigma(4) = 13$ и $\Sigma(5) = 4098$. Нижняя оценка для $\Sigma(6)$ выше башни из пятнадцати экспонент.

Гипервычисления

Гипервычисления — спекуляции о машинах, выходящих за тезис: бесконечное ускорение, оракулы в физике, точные действительные числа за один шаг.

Тезис Чёрча–Тьюринга — не теорема, и физика не обязана подчиняться определению.

Но за девяносто лет ни одна реализация не вышла за класс тьюринг-вычислимого: гипотез хватает, работающих машин — нет.

Десятая проблема Гильберта

Неразрешимость живёт не только в вопросах о программах — целочисленные уравнения упираются в ту же границу.

Определение 11 (Диофантово уравнение)

Уравнение $p(x_1, ..., x_n) = 0$, где p — многочлен с целыми коэффициентами, называется **диофантовым**.

Спросить: существует ли набор целых значений, обращающий многочлен в ноль?

Пример (Разное поведение уравнений)

$x^2 + y^2 = z^2$ разрешимо: $3^2 + 4^2 = 5^2$.

$x^2 + y^2 = 3$ неразрешимо: квадрат целого числа даёт остаток 0 или 1 по модулю 4, и сумма двух квадратов не даёт 3.

Гильберт в 1900 году просил алгоритм, разделяющий уравнения этих двух сортов.

Неразрешимость десятой проблемы

Разделить разрешимые и неразрешимые уравнения тоже не выходит.

Теорема 12 (Десятая проблема Гильберта (MRDP, 1970))

Не существует алгоритма, определяющего по диофантову уравнению, есть ли у него решение в целых числах.

Основа — теорема MRDP: диофантовы множества — это в точности перечислимые.

Доказательство

Докажем сведением: пары (m, w) с остановкой машины m на w перечислимы.

По теореме MRDP это множество диофантово: уравнение $p(m, w, y_1, ..., y_k) = 0$ разрешимо ровно при остановке m на w.

Решатель диофантовых уравнений решал бы проблему остановки — противоречие.

Что с этим делать инженеру?

Неразрешимость не запрещает практику — она задаёт её форму: инструменты работают на фрагментах и приближениях.

Если задача неразрешима на машине Тьюринга, её не решит ни один язык программирования.

Значит: искать разрешимые фрагменты, ограничивать класс входов, строить приближения.

Граница вычислимого не зависит от модели: динамические языки, нейросети и квантовые вычисления её не сдвигают.

Что мы поняли?

Три класса — три уровня гарантий: полный ответ, гарантия для своих слов, гарантия для чужих.

Диагональ, сведения и теорема Райса провели границу.

- Разрешитель отвечает на каждом входе, распознаватель — только для своих слов, ко-распознаватель — только для чужих.
- Перечислитель равносилен распознавателю, разрешимость — перечислимость с двух сторон, а дополнение HALT показывает, что второй стороны может не быть.
- Диагональ запрещает решатель остановки, сведения разносят неразрешимость: пустота, эквивалентность, домино Поста.
- Теорема Райса закрывает нетривиальные свойства языка — отсюда пределы анализаторов.
- Оракулы и десятая проблема Гильберта переносят границу вне программ — в целочисленные уравнения.

Вопросы для самопроверки

Вопросы для самопроверки — о разрешимости и теореме Райса

1. Чем разрешитель отличается от распознавателя и ко-распознавателя? Какой из классов замкнут относительно дополнения? Почему два других класса этим свойством не обладают?
2. Восстановите оба перехода в равносильности перечислителя и распознавателя.
3. Воспроизведите диагональ: как машина D использует H и где в рассуждении появляется собственный код?
4. Сведите пустоту языка к эквивалентности программ: какую машину ставит рядом сводящая функция?
5. Опишите набор костей Поста для пары $(\langle M \rangle, w)$ и то, что читается в совпадении.
6. Применима ли теорема Райса к свойству «у машины ровно три состояния»? Почему свойство «останавливается ли M на пустом входе» вне её досягаемости?

Что почитать

Чтение из главы m26 закрывает теорему Райса и разделы за пределами машин Тьюринга.

- m26 — «Теорема Райса», «За пределами машин Тьюринга».